# Apprentissage par Renforcement sur Highway-v0

## DQN Custom vs Stable-Baselines3

Groupe 17

Antoine Yezou, Maxence Rossignol, Zacharie Boumard, Ylias Larbi

Avril 2026

# Sommaire


1. Environnement et Modélisation
2. DQN : Algorithme et Implémentation
3. Résultats et Comparaison
4. Extension et Conclusion

# L'Environnement : Highway-v0

**Le problème :** conduire sur une autoroute à 4 voies parmi **45 véhicules** pendant 30 secondes, en maximisant la vitesse sans collision.

**Configuration :**

- 4 voies, 45 véhicules, durée 30s
- Vitesses cibles : $[20, 25, 30]$ m/s
- Récompense normalisée
- Collision $\Rightarrow$ fin d'épisode

**5 actions discrètes :**

1. Changer à gauche
2. Idle (maintenir)
3. Changer à droite
4. Accélérer
5. Ralentir

*Actions de type* `DiscreteMetaAction` *: un contrôleur bas niveau gère la trajectoire.*

# État et Récompense

**Observation (état $s_t$) :** matrice cinématique $10 \times 5$

| | présence | $x$ | $y$ | $v_x$ | $v_y$ |
|---|---|---|---|---|---|
| Ego | 1.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| Voisin 1 | 1.0 | $-0.3$ | 0.0 | $-0.2$ | 0.0 |
| Voisin 2 | 1.0 | 0.1 | 0.25 | 0.05 | 0.0 |
| ... | ... | ... | ... | ... | ... |

Valeurs **relatives** au véhicule ego, **normalisées** entre $[-1, 1]$.

**Récompense à chaque pas :**

- Vitesse élevée : $+0.7$ (progressif sur $[22, 30]$ m/s)
- Collision : $-1.5$ (fin d'épisode)
- Changement de voie : $-0.02$

### Arbitrage

L'agent doit maximiser sa vitesse sans s'exposer au coût de collision ($\sim 2\times$ le bonus de vitesse).

# DQN : Principe

**Idée :** apprendre une fonction $Q(s, a)$ qui prédit le **cumul de récompenses futures** pour chaque action.

**Boucle d'apprentissage :**

1. **Observer** l'état $s_t$ (matrice $10 \times 5$ aplatie en vecteur de 50)
2. **Choisir** une action : aléatoire (probabilité $\epsilon$) ou $\arg\max_a Q(s_t, a)$
3. **Exécuter** l'action, observer $r_t$, $s_{t+1}$
4. **Stocker** la transition $(s_t, a_t, r_t, s_{t+1}, done)$ dans le *replay buffer*
5. **Apprendre** sur un batch aléatoire de 64 transitions

**Cible d'apprentissage :**

$$y = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a') \quad \text{avec } \gamma = 0.99$$

L'agent minimise $\mathcal{L} = (Q(s_t, a_t) - y)^2$ par descente de gradient.

# Composants clés du DQN

**Experience Replay**

- Buffer circulaire de 50 000 transitions
- Apprentissage sur des transitions **aléatoires** (pas consécutives)
- Casse les corrélations temporelles $\Rightarrow$ stabilise l'entraînement

**Exploration ($\epsilon$-greedy)**

- $\epsilon$ : 1.0 $\rightarrow$ 0.05 (linéaire)
- Décroissance sur 50% des timesteps
- 5% d'exploration résiduelle

**Target Network**

- Copie **gelée** du réseau
- Mise à jour tous les 500 pas
- Fournit des cibles stables pour éviter la divergence des Q-values

**Réseau (MLP)**

- Entrée : 50 (matrice aplatie)
- Couches : 256 $\rightarrow$ 256 (ReLU)
- Sortie : 5 Q-values
- Gradient clippé à 1.0

# Protocole expérimental

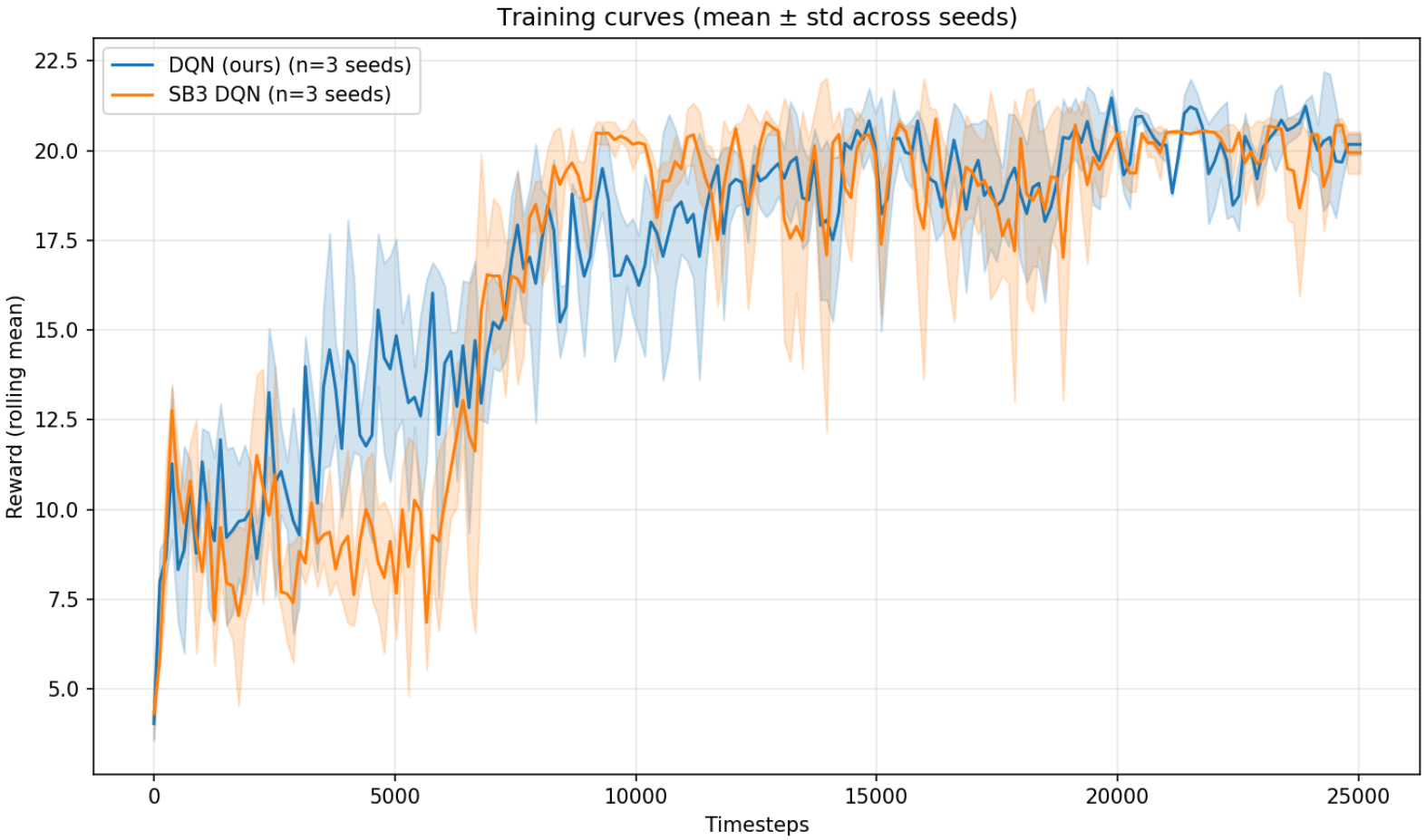
**Comparaison équitable :** même nombre de timesteps, même config, mêmes seeds.

| | DQN (custom) | SB3 DQN |
|---|---|---|
| Timesteps | 40 000 | 40 000 |
| Seeds | 52, 53, 54 | 52, 53, 54 |
| Learning rate | $10^{-3}$ | $10^{-3}$ |
| Buffer | 50 000 | 50 000 |
| Batch size | 64 | 64 |
| Target update | 500 steps | 500 steps |
| $\epsilon$ schedule | $1 \to 0.05$ sur 50% | $1 \to 0.05$ sur 30% |
| Réseau | MLP 256-256 | MLP 256-256 |

Évaluation : **50 épisodes** par seed, politique déterministe ($\epsilon = 0$).

Entraîné sur le cluster DCE de CentraleSupélec (NVIDIA GTX 1080 Ti, CUDA 12.4).

# Courbes d'entraînement




Moyenne ± écart-type sur 3 seeds (architectures identiques 256-256). Lissage par moyenne glissante (fenêtre 10).

- Notre DQN converge vite (∼5k timesteps) mais reste bruité (crashes).

# Évaluation quantitative (50 runs × 3 seeds)

| Agent | Seed | Reward moy. | Std | Crashes |
|---|---|---|---|---|
| DQN (custom) | 52 | 19.81 | 4.52 | 9/50 (18%) |
| DQN (custom) | 53 | 18.99 | 5.51 | 12/50 (24%) |
| DQN (custom) | 54 | 20.76 | 3.44 | 8/50 (16%) |
| **DQN (custom)** | **total** | **19.85** | **4.63** | **29/150 (19%)** |
| SB3 DQN | 52 | 20.45 | 0.00 | 0/50 (0%) |
| SB3 DQN | 53 | 20.55 | 0.05 | 0/50 (0%) |
| SB3 DQN | 54 | 20.45 | 0.00 | 0/50 (0%) |
| **SB3 DQN** | **total** | **20.48** | **0.05** | **0/150 (0%)** |

SB3 : reward légèrement supérieure (+3%), variance quasi-nulle, et **0 crash** vs 19% pour notre DQN.

# Analyse des modes d'échec

## Notre DQN (19% de crashes)

Deux patterns identifiés :

- **Crash précoce** (steps 4–8) : changement de voie dans une voie occupée juste après le reset. Reward très faible (∼2–6).
- **Crash tardif** (steps 19–30) : collision en fin d'épisode dans du trafic dense. Reward proche du max (∼20).

## SB3 DQN (0% de crashes)

Politique ultra-conservative :

- Reward quasi-constante (∼20.45)
- Zéro crash sur 150 épisodes
- Probablement : vitesse constante, pas de changement de voie
- Sûr mais ne maximise pas la reward (pics à 24+ pour notre DQN)

### Compromis sécurité vs performance

SB3 privilégie la sécurité (0 crash) au détriment du reward maximal. Notre DQN tente des manœuvres risquées qui parfois échouent mais atteignent des rewards plus élevées.

# Choix de design

- **Timesteps comme unité commune :** permet une comparaison directe DQN vs SB3 sur le même axe. L'ancienne version utilisait des épisodes (DQN) vs timesteps (SB3) $\Rightarrow$ incomparable.
- $\gamma = 0.99$ **:** horizon lointain nécessaire pour anticiper le trafic. Un $\gamma$ plus faible rendrait l'agent trop myope (ne planifie pas les dépassements).
- **MLP 256-256 identique :** même architecture pour les deux agents. La v2 utilisait le `MlpPolicy` par défaut (64-64) pour SB3 $\Rightarrow$ comparaison biaisée. La v3 corrige ce problème.
- **Sauvegarde incrémentale (CSV) :** résultats enregistrés après chaque seed. A sauvé nos données quand le premier job cluster a été tué par SLURM.

# Extension : *À compléter*

**Direction envisagée :** *[Ex : Double-DQN / Prioritized Experience Replay / Safety-RL]*

**Hypothèse :**

- *[Ex : Double-DQN réduit la surestimation des Q-values ⇒ politique plus stable]*

**Protocole expérimental :**

- *[Comparaison / ablation / variation contrôlée]*

**Résultats :**

- *[À compléter]*

# Conclusion

**Résultats principaux :**

- Avec des architectures identiques (256-256), **SB3 surpasse notre DQN** : reward 20.48 vs 19.85, 0% vs 19% de crashes.
- SB3 apprend une politique conservatrice très stable. Notre DQN est plus agressif (pics à 24+) mais moins fiable.

**Leçons apprises :**

- L'importance d'une comparaison équitable : même architecture, mêmes timesteps, mêmes seeds. La v2 (architectures différentes) donnait des conclusions **opposées**.
- La robustesse de l'infrastructure (sauvegardes incrémentales, gestion du cluster).
- Les bugs silencieux (Monitor wrapper, observation space) peuvent invalider des résultats sans signal d'erreur.

**Perspectives :**

- Extension (Double-DQN, PER, Safety-RL)
- Réduire le taux de crash de notre DQN (reward shaping, exploration)
- Généralisation à d'autres environnements (intersection, rond-point)